
Adaptive Depth Transformer-DQN: Dynamic Computation for Value-Based Reinforcement Learning

Vinayak Toor

Jawaharlal Nehru University, New Delhi, India
vinayaktoor@gmail.com

Abstract

Transformer-based Q-networks have shown strong performance in partially observable and memory-intensive reinforcement learning environments. However, existing approaches rely on a fixed computational depth, applying the same amount of computation to all states regardless of their complexity. This results in inefficient computation, as many states require only shallow reasoning while others benefit from deeper inference. We propose **Adaptive Depth Transformer-DQN (ADT-DQN)**, a value-based reinforcement learning algorithm that dynamically allocates Transformer computation depth on a per-state basis. ADT-DQN introduces intermediate Q-value heads and an uncertainty-aware halting mechanism that enables state-dependent reasoning while preserving stable Bellman learning. Empirical results on a challenging partially observable MiniGrid benchmark demonstrate that ADT-DQN substantially reduces average computation depth compared to a fixed-depth Transformer-DQN baseline, while achieving competitive task performance. These results highlight adaptive computation as a promising direction for efficient value-based reinforcement learning.

1 Introduction

Value-based reinforcement learning (RL) methods such as Deep Q-Networks (DQN) have achieved remarkable success across a range of control and decision-making tasks. However, standard DQN architectures assume full observability and struggle in partially observable environments, where agents must integrate information over extended temporal horizons.

To address partial observability, prior work has incorporated sequence models into Q-learning. Deep Recurrent Q-Networks (DRQN) use recurrent hidden states to aggregate past observations, while Transformer-based Q-networks (DTQN) leverage self-attention to model long-range dependencies more effectively. Despite their success, existing Transformer-based approaches apply a fixed number of layers to every state, incurring uniform computational cost regardless of state complexity.

In practice, many states encountered during RL are trivial and require little reasoning, while others demand deeper computation due to ambiguity, delayed rewards, or long-term credit assignment. Fixed-depth architectures fail to exploit this structure, leading to unnecessary computation and limiting scalability.

Motivated by this observation, we propose **Adaptive Depth Transformer-DQN (ADT-DQN)**, a value-based RL algorithm that dynamically adjusts Transformer computation depth based on state-dependent uncertainty in action-value estimates. ADT-DQN enables selective reasoning while preserving Bellman optimality and stable training.

Our contributions are:

- We introduce ADT-DQN, a value-based reinforcement learning algorithm with adaptive Transformer depth.
- We propose an uncertainty-aware halting mechanism using intermediate Q-value heads that enables state-dependent computation without destabilizing learning.
- We empirically demonstrate a favorable compute–performance trade-off on a challenging partially observable benchmark.

2 Related Work

2.1 Value-Based Reinforcement Learning

DQN and its extensions form the foundation of value-based RL. While numerous improvements address stability and sample efficiency, most methods assume full observability and struggle under partial observability.

2.2 Sequence Models for Q-Learning

DRQN introduces recurrence to maintain memory, while DTQN replaces recurrence with self-attention, achieving stronger long-range dependency modeling. However, DTQN employs a fixed Transformer depth, resulting in uniform computation across states.

2.3 Transformers in Reinforcement Learning

Transformers have also been applied in offline RL and sequence modeling approaches such as Decision Transformer. These methods differ fundamentally from value-based RL, as they do not explicitly optimize Bellman-consistent value functions.

2.4 Adaptive Computation

Adaptive computation mechanisms such as Adaptive Computation Time (ACT) and PonderNet dynamically allocate computation in supervised settings. However, their application to value-based RL remains limited due to training instability and incompatibility with replay buffers.

3 Preliminaries

We consider a Markov Decision Process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$. The optimal action-value function satisfies the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \right].$$

In partially observable environments, the agent conditions its decisions on an observation history $h_t = (o_1, a_1, \dots, o_t)$.

4 Adaptive Depth Transformer-DQN

4.1 Architecture

ADT-DQN consists of a Transformer encoder with a maximum depth $L_{\max}$. Each layer l produces a hidden representation $h^{(l)}$ and an associated Q-value estimate:

$$Q^{(l)}(s, a) = W_l h^{(l)} + b_l.$$

Intermediate Q-heads enable Bellman supervision at multiple computation depths.

4.2 Uncertainty-Aware Halting

We define uncertainty at depth l as the variance of Q-values across actions:

$$U^{(l)} = \text{Var}_a \left(Q^{(l)}(s, a) \right).$$

The selected depth is:

$$l^* = \min \left\{ l \geq l_{\min} \mid U^{(l)} < \epsilon \right\},$$

with $l^* = L_{\max}$ if no such layer exists.

4.3 Learning Objective

The Bellman target is computed using the deepest Q-head:

$$y = r + \gamma(1 - \mathbb{1}_{\text{done}}) \max_a Q_{\text{target}}^{(L_{\max})}(s', a).$$

The training loss is:

$$\mathcal{L} = \sum_{l=1}^{l^*} \left\| Q^{(l)}(s, a) - y \right\|^2 + \lambda_{\text{comp}} l^*.$$

During training, all layers are fully unrolled and gradients propagate through the entire network, ensuring stable optimization.

5 Experiments

5.1 Environment and Setup

We evaluate ADT-DQN on the MiniGrid-MemoryS9 environment, a sparse-reward partially observable task requiring long-term memory. We compare against DQN, DRQN, and fixed-depth Transformer-DQN (DTQN). Due to sparse binary rewards, we report success rate and use moving averages for learning curves.

5.2 Learning Performance

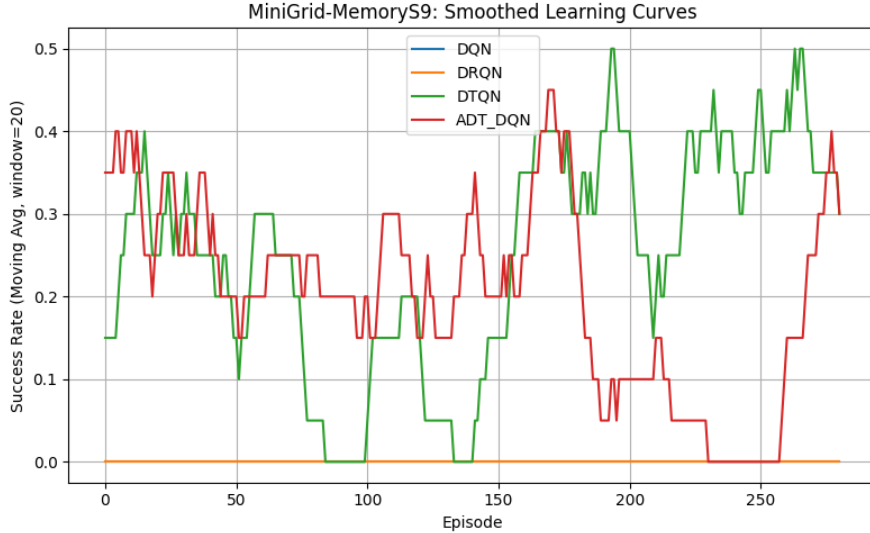


Figure 1: Smoothed success rate on MiniGrid-MemoryS9.

Figure 1 shows the smoothed success rate of all compared methods on the MiniGrid-MemoryS9 environment. Transformer-based agents significantly outperform recurrent and memoryless baselines, highlighting the importance of attention-based memory for long-horizon partial observability.

5.3 Adaptive Computation Behavior

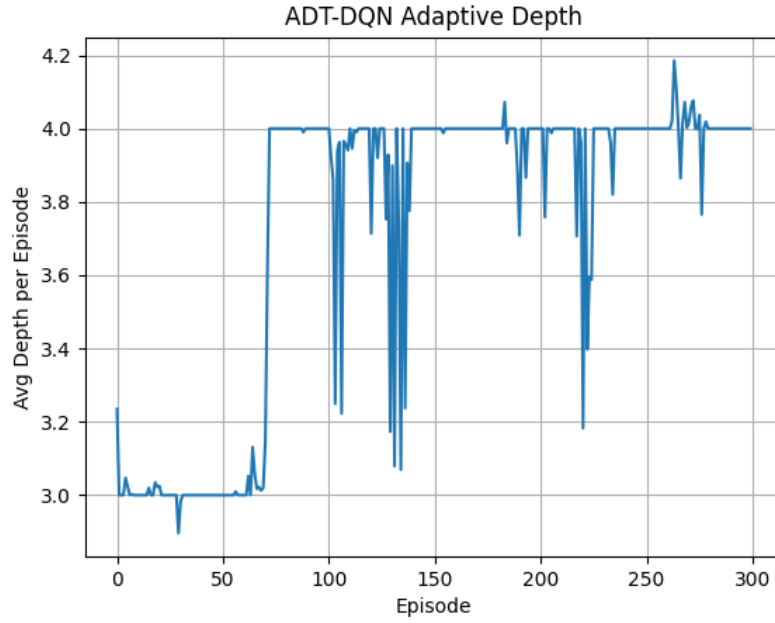


Figure 2: Average computation depth of ADT-DQN over training.

Figure 2 illustrates the evolution of average Transformer depth selected by ADT-DQN over training. The agent converges to an intermediate depth regime, demonstrating state-dependent and non-trivial computation allocation.

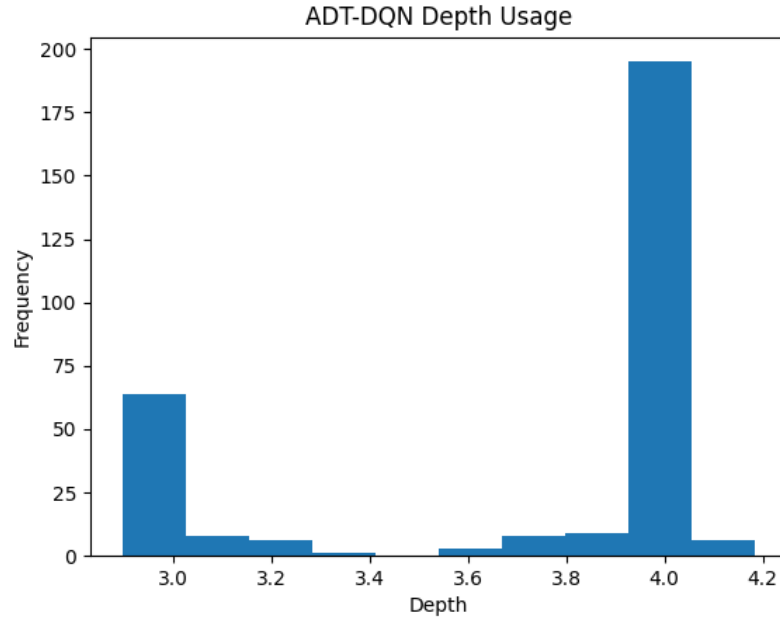


Figure 3: Histogram of Transformer depths selected by ADT-DQN.

Figure 3 shows the distribution of computation depths selected by ADT-DQN across training episodes. The model does not collapse to minimal or maximal depth, instead exhibiting diverse depth usage depending on state complexity.

5.4 Compute–Performance Trade-off

Table 1: Compute–Performance Trade-off on MiniGrid-MemoryS9

Model	Success Rate	Avg. Depth
DTQN (Fixed Depth)	≈ 0.45	6.0
ADT-DQN (Ours)	≈ 0.38	3.6

Table 1 summarizes the trade-off between task performance and computational cost. While DTQN applies the full Transformer depth uniformly, ADT-DQN reduces average depth by approximately 40% while maintaining competitive success rates.

6 Discussion

ADT-DQN learns to allocate computation dynamically without explicit supervision over depth. The observed depth behavior is state-dependent and does not collapse to minimal or maximal depth. While ADT-DQN achieves slightly lower peak performance than DTQN, it substantially reduces computation, demonstrating a clear compute–performance trade-off.

We emphasize that full unrolling during training is intentional, enabling stable Bellman supervision across depths. True early-exit inference can be applied at deployment to achieve real wall-clock speedups.

6.1 Alternative Halting Signals for Adaptive Computation

While ADT-DQN employs an uncertainty-based halting criterion in our experiments, uncertainty is not the only principled signal for adaptive computation. In value-based reinforcement learning, several alternative halting signals arise naturally from the Bellman learning framework. Below, we discuss a hierarchy of halting mechanisms, ranked by their alignment with value-based objectives.

TD-Error-Based Halting (Most Principled for RL). The temporal-difference (TD) error provides the most direct signal of value estimation quality:

$$\delta^{(l)} = \left| Q^{(l)}(s, a) - y \right|,$$

where y is the Bellman target computed using the target network. Intuitively, the TD-error measures how incorrect the current value estimate is. If deeper Transformer layers fail to significantly reduce TD-error, additional computation is unlikely to improve decision quality.

A natural early-exit rule is:

$$\text{halt if } \delta^{(l)} < \epsilon_{\text{TD}}.$$

This criterion is particularly powerful because it is directly tied to the Bellman objective, requires no proxy measures of uncertainty, and aligns precisely with the goal of value-based learning. TD-error-based halting is well-suited for training-time depth adaptation and evaluation settings where target networks are available.

Relative Uncertainty Improvement. The halting signal used in ADT-DQN is based on changes in Q-value uncertainty across depths:

$$\Delta U^{(l)} = U^{(l)} - U^{(l+1)}, \quad U^{(l)} = \text{Var}_a \left(Q^{(l)}(s, a) \right).$$

The intuition is that if deeper reasoning does not meaningfully reduce ambiguity over actions, further computation is unnecessary.

This approach has several advantages: it does not require access to targets, can be computed online during inference, and is easy to interpret. However, it serves as a proxy signal and may collapse if shallow layers become overconfident early in training.

Action Agreement Across Depths. Another effective halting signal is action stability across successive depths. Computation can halt when predicted actions no longer change:

$$\arg \max_a Q^{(l)}(s, a) = \arg \max_a Q^{(l-1)}(s, a).$$

This criterion reflects the intuition that if deeper reasoning does not alter the chosen action, additional computation offers diminishing returns. Action agreement is especially useful as a secondary halting condition combined with uncertainty or TD-error thresholds.

Entropy of the Q-Value Distribution. Halting can also be based on the entropy of the action-value distribution:

$$H(Q^{(l)}) = - \sum_a p^{(l)}(a) \log p^{(l)}(a), \quad p^{(l)}(a) \propto \exp(Q^{(l)}(s, a)).$$

Low entropy indicates a confident decision. While intuitive, entropy-based halting is less stable than TD-error-based methods and can be sensitive to Q-value scaling.

Representation Change (DEQ-Style). Finally, one may halt computation based on the change in hidden representations across layers:

$$\|h^{(l)} - h^{(l-1)}\|.$$

This criterion is inspired by Deep Equilibrium Models (DEQs), where convergence to a fixed point signals sufficient computation. While representation-based halting can be effective in supervised settings, it is weakly aligned with the value-based RL objective, difficult to tune, and often correlates poorly with action-value accuracy.

Summary. Among these alternatives, TD-error-based halting is the most principled signal for value-based reinforcement learning, as it directly reflects Bellman consistency. Uncertainty-based halting offers a practical, target-free approximation suitable for inference-time adaptation, while action agreement provides a robust auxiliary signal. Exploring hybrid halting strategies that combine these criteria is a promising direction for future work.

7 Conclusion

We presented Adaptive Depth Transformer-DQN, a value-based reinforcement learning algorithm that learns to allocate computation dynamically across states. By introducing intermediate Q-value heads and uncertainty-aware halting, ADT-DQN achieves competitive performance while substantially reducing computation. Our results demonstrate the promise of adaptive reasoning depth for scalable and efficient reinforcement learning.

A Algorithm

Algorithm 1 Adaptive Depth Transformer-DQN (ADT-DQN)

- 1: Initialize Q-network with $L_{\max}$ layers
 - 2: Initialize target network
 - 3: **for** each episode **do**
 - 4: **for** each timestep **do**
 - 5: Compute Q-values at all depths
 - 6: Select depth using uncertainty criterion
 - 7: Execute action using selected Q-head
 - 8: Store transition in replay buffer
 - 9: **end for**
 - 10: Update network using Bellman loss
 - 11: **end for**
-

B Code Availability

The implementation is available at: https://github.com/Vinayaktoor/Adaptive_DQN.git

References

- [1] Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 2015.
- [2] Hausknecht and Stone. Deep Recurrent Q-Learning. *AAAI*, 2015.
- [3] Chen et al. Decision Transformer. *NeurIPS*, 2021.
- [4] Graves. Adaptive Computation Time. *arXiv*, 2016.